

For single inheritance, determining the MRO is easy: just make a chain of parent classes. For multiple inheritance, it's trickier. Python's MRO follows the C3 algorithm, whose details are beyond the scope of this book. But you can determine the MRO by remembering two rules:

- Python checks child classes before parent classes.
- Python checks inherited classes listed left to right in the `class` statement.

If we call `getBoardStr()` on a `HybridBoard` object, Python checks the `HybridBoard` class first. Then, because the class's parents from left to right are `HintBoard` and `MiniBoard`, Python checks `HintBoard`. This parent class has a `getBoardStr()` method, so `HybridBoard` inherits and calls it.

But it doesn't end there: next, this method calls `super().getBoardStr()`. *Super* is a somewhat misleading name for Python's `super()` function, because it doesn't return the parent class but rather the next class in the MRO. This means that when we call `getBoardStr()` on a `HybridBoard` object, the next class in its MRO, after `HintBoard`, is `MiniBoard`, not the parent class `TTTBoard`. So the call to `super().getBoardStr()` calls the `MiniBoard` class's `getBoardStr()` method, which returns the miniature tic-tac-toe board string. The remaining code in the `HintBoard` class's `getBoardStr()` after this `super()` call appends the hint text to this string.

If we change the `HybridBoard` class's `class` statement so it lists `MiniBoard` first and `HintBoard` second, its MRO will put `MiniBoard` before `HintBoard`. This means `HybridBoard` inherits `getBoardStr()` from `MiniBoard`, which doesn't have a call to `super()`. This ordering is what caused the bug that made the miniature tic-tac-toe board display without hints: without a `super()` call, the `MiniBoard` class's `getBoardStr()` method never calls the `HintBoard` class's `getBoardStr()` method.

Multiple inheritance allows you to create a lot of functionality in a small amount of code but easily leads to overengineered, hard-to-understand code. Favor single inheritance, mixin classes, or no inheritance. These techniques are often more than capable of carrying out your program's tasks.

Summary